



UNIVERSIDADE FEDERAL DO PARÁ

Faculdade de Computação — FCOMP

Disciplina de Inteligência Artificial

Trabalho Prático 2 — Metaheurísticas

Problema das 8 Rainhas: GRASP e Algoritmo Genético

Professor: Dr. Jefferson Moraes

Discente: Andrey Júlio da Silva Rocha.

Resumo

Este relatório apresenta a implementação, a análise experimental e a comparação crítica de dois algoritmos metaheurísticos aplicados ao Problema das 8 Rainhas: o GRASP (Greedy Randomized Adaptive Search Procedure) e o Algoritmo Genético (AG). Ambos os algoritmos foram executados 50 vezes cada, com os experimentos automatizados por meio da plataforma n8n, garantindo reprodutibilidade e coleta estruturada de métricas. Os resultados demonstram a superioridade do GRASP em todos os indicadores avaliados para este problema específico, alcançando 100% de taxa de sucesso com tempo médio de 13,22 ms, enquanto o AG obteve 76% de taxa de sucesso e tempo médio de 217,32 ms. A análise discute as causas estruturais dessas diferenças, as limitações de cada abordagem e possíveis melhorias futuras.

Parte 1 — Modelagem do Problema

1.1 Contextualização

O Problema das 8 Rainhas é um clássico da inteligência artificial e da teoria da computação, consistindo em posicionar oito rainhas em um tabuleiro de xadrez 8×8 de forma que nenhuma rainha ataque outra. Uma rainha ataca outra quando compartilham a mesma linha, coluna ou diagonal. O número total de configurações possíveis para posicionar 8 rainhas em 64 casas é $C(64,8) \approx 4,4$ bilhões, tornando a busca exaustiva inviável do ponto de vista prático e justificando o uso de metaheurísticas.

1.2 Representação do Estado

A representação adotada é um vetor unidimensional de 8 posições, denotado como $Q = [q_0, q_1, q_2, q_3, q_4, q_5, q_6, q_7]$, onde:

- O índice i ($0 \leq i \leq 7$) representa a coluna do tabuleiro;
- O valor q_i ($0 \leq q_i \leq 7$) representa a linha em que a rainha da coluna i está posicionada.

Essa representação garante implicitamente que não há conflitos de coluna, pois cada coluna possui exatamente uma rainha. Por exemplo, o vetor $[6,1,5,2,0,3,7,4]$ indica que a rainha da coluna 0 está na linha 6, a da coluna 1 na linha 1, e assim por diante. O espaço de estados possíveis com esta representação é de $8^8 = 16.777.216$ configurações, com 92 soluções ótimas conhecidas.

1.3 Função Objetivo e Função de Custo

A função objetivo é a minimização do número de conflitos entre pares de rainhas. Dois tipos de conflito são verificados:

- Conflito horizontal: duas rainhas na mesma linha, ou seja, $q_i = q_j$ para $i \neq j$;
- Conflito diagonal: duas rainhas na mesma diagonal, ou seja, $|q_i - q_j| = |i - j|$.

A função de custo $C(Q)$ é definida formalmente como:

$$C(Q) = \sum_i \sum_{j>i} [\text{conflito}(i, j)]$$

onde $\text{conflito}(i, j) = 1$ se $q_i = q_j$ (conflito de linha) ou $|q_i - q_j| = |i - j|$ (conflito de diagonal), e 0 caso contrário. Uma solução ótima satisfaz $C(Q) = 0$.

1.4 Espaço de Busca e Critério de Parada

O espaço de busca compreende todas as permutações possíveis do vetor Q . O critério de parada geral é satisfeito quando:

- A função de custo atinge $C(Q) = 0$, indicando solução ótima encontrada; ou
- O número máximo de iterações (GRASP: 1.000 iterações) ou gerações (AG: 1.000 gerações) é atingido.

1.5 Estratégia Geral dos Algoritmos

Dois algoritmos foram implementados com estratégias distintas de busca:

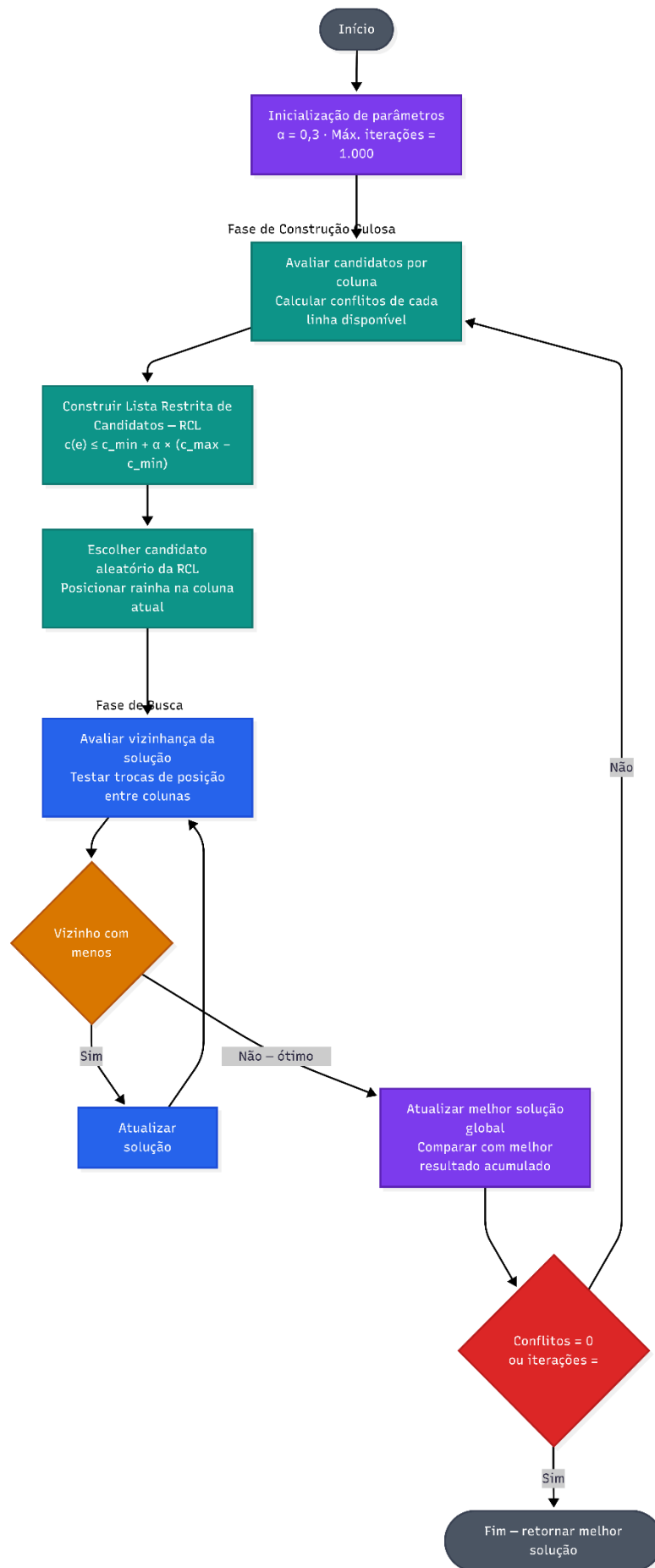
- GRASP: realiza uma construção gulosa randomizada da solução inicial seguida de busca local iterativa, combinando diversificação e intensificação em cada iteração independente.
- Algoritmo Genético: mantém uma população de soluções candidatas que evoluem ao longo de gerações por meio de operadores de seleção, cruzamento e mutação, explorando o espaço de busca por múltiplas trajetórias simultâneas.

Parte 2 — Implementação do GRASP

2.1 O Algoritmo GRASP

O GRASP (Greedy Randomized Adaptive Search Procedure) é uma metaheurística iterativa multistart proposta por Feo e Resende (1995). Cada iteração do GRASP é composta por duas fases distintas e complementares: (1) uma fase de construção gulosa randomizada, que gera uma solução inicial de boa qualidade de forma eficiente; e (2) uma fase de busca local, que melhora iterativamente essa solução explorando sua vizinhança. A melhor solução encontrada ao longo de todas as iterações é mantida como resultado final.

A principal vantagem do GRASP é sua capacidade de equilibrar diversificação (exploração de regiões distintas do espaço de busca por meio da aleatoriedade na construção) e intensificação (melhoria sistemática pela busca local), sem requerer estruturas de memória complexas como ocorre em outras metaheurísticas.



2.2 Parâmetros Utilizados

Parâmetro	Valor
N (número de rainhas)	8
Alpha (α)	0,3
Limite de iterações	1.000
Execuções	50

2.3 Fase de Construção Gulosa Randomizada

A construção da solução inicial ocorre coluna por coluna, do índice 0 ao índice 7. Para cada coluna i , é avaliado o conjunto completo de candidatos, composto pelas 8 posições de linha ainda disponíveis (linhas não ocupadas pelas rainhas já posicionadas nas colunas anteriores). Cada candidato é avaliado por uma função gulosa que computa o número de conflitos que ele geraria com as rainhas já posicionadas, considerando tanto conflitos de linha quanto de diagonal.

O conjunto de candidatos é então ordenado de forma crescente pelo número de conflitos. A Lista Restrita de Candidatos (RCL) é formada pelos candidatos cujo custo $c(e)$ satisfaz a condição:

$$c(e) \leq c_{\min} + \alpha \times (c_{\max} - c_{\min})$$

onde $c_{\min}$ é o menor custo observado, $c_{\max}$ é o maior custo, e $\alpha = 0,3$ é o parâmetro de aleatoriedade. Com $\alpha = 0,3$, apenas os 30% superiores do intervalo de variação de custo são admitidos na RCL, o que garante que somente candidatos próximos ao ótimo guloso local integrem a lista, mantendo a qualidade da construção. Um candidato é então escolhido aleatoriamente e de forma uniforme dentre os elementos da RCL, introduzindo diversidade à solução sem comprometer a qualidade geral da construção.

2.4 Fase de Busca Local

Após a construção da solução inicial, a fase de busca local percorre sistematicamente a vizinhança da solução atual. A estrutura de vizinhança adotada consiste na troca de posições

(linhas) entre duas rainhas de colunas distintas. Para cada par de colunas (i, j) , é avaliado se a troca $q_i \leftrightarrow q_j$ reduz o número de conflitos da solução. Caso uma melhora seja encontrada (estratégia de melhora imediata, first improvement), a troca é realizada e o processo recomeça a partir da nova solução. A busca local termina quando nenhuma troca na vizinhança produz melhora, caracterizando um ótimo local.

A fase de busca local, por sua natureza determinística de melhora, converge rapidamente para ótimos locais. Para o Problema das 8 Rainhas com $N=8$, a densidade de ótimos globais no espaço de busca é relativamente alta, favorecendo a convergência do GRASP para a solução ótima em poucas iterações.

2.5 Resultados Obtidos pelo GRASP

O GRASP foi executado 50 vezes, obtendo os seguintes resultados:

- Taxa de sucesso: 100% — todas as 50 execuções encontraram uma solução com 0 conflitos.
- Tempo médio de execução: 13,22 ms (desvio padrão: 12,29 ms).
- Número médio de iterações até convergência: 6,9 (desvio padrão: 6,43).
- Conflitos médio final: 0,00 (solução ótima em todas as execuções).

As cinco melhores soluções distintas encontradas foram:

Rank	Vetor de posicionamento Q
1°	[6, 1, 5, 2, 0, 3, 7, 4]
2°	[3, 1, 6, 2, 5, 7, 4, 0]
3°	[5, 3, 1, 7, 4, 6, 0, 2]
4°	[2, 5, 7, 0, 4, 6, 1, 3]
5°	[6, 1, 3, 0, 7, 4, 2, 5]

2.6 Vantagens e Limitações do GRASP

Vantagens observadas:

- Convergência perfeita: 100% de taxa de sucesso em 50 execuções independentes.
- Altíssima eficiência temporal: tempo médio de apenas 13,22 ms, tornando-o adequado para aplicações com restrições de tempo.

- Baixa complexidade de implementação: não requer estruturas de dados complexas ou ajuste fino de múltiplos hiperparâmetros.
- Convergência ultrarrápida: em média, apenas 6,9 iterações foram necessárias para encontrar a solução ótima.

Limitações observadas:

- Escalabilidade: para instâncias maiores do problema ($N \gg 8$), a busca local pode se tornar computacionalmente cara e a qualidade das soluções pode degradar.
- Sensibilidade ao alpha: a escolha inadequada de α pode comprometer tanto a qualidade das soluções construídas quanto a diversidade explorada.
- Ausência de memória: o GRASP não acumula informação entre iterações, podendo revisitar regiões já exploradas do espaço de busca.

Parte 3 — Implementação do Algoritmo Genético

3.1 Fundamentos do Algoritmo Genético

O Algoritmo Genético (AG) é uma metaheurística baseada nos princípios da seleção natural e evolução biológica, proposta originalmente por John Holland (1975). O AG mantém uma população de soluções candidatas (indivíduos) que evoluem ao longo de gerações por meio da aplicação de operadores genéticos: seleção, cruzamento e mutação. A ideia central é que os indivíduos mais adaptados (com melhor fitness) têm maior probabilidade de transmitir seu material genético às gerações seguintes, convergindo progressivamente para soluções de melhor qualidade.

3.2 Codificação Binária

A representação de cada indivíduo utiliza codificação binária. Cada rainha ocupa 3 bits (suficientes para representar valores de 0 a 7, já que $2^3 = 8$), resultando em um cromossomo de 24 bits para as 8 rainhas. Por exemplo, a solução $[6, 1, 5, 2, 0, 3, 7, 4]$ é codificada como:

Col.0	Col.1	Col.2	Col.3	Col.4	Col.5	Col.6	Col.7
6→110	1→001	5→101	2→010	0→000	3→011	7→111	4→100

O cromossomo resultante é: 110 001 101 010 000 011 111 100 (24 bits no total).

3.3 Função Fitness

A função fitness é calculada a partir do número de conflitos $C(Q)$ da solução decodificada. A fórmula adotada transforma o problema de minimização em maximização:

$$\text{fitness}(Q) = 1 / (1 + C(Q))$$

Quando $C(Q) = 0$ (solução ótima), $\text{fitness} = 1,0$, que é o valor máximo possível. Quanto maior o número de conflitos, menor o fitness. Essa transformação é necessária para a operação de seleção por roleta, que exige valores positivos proporcionais à qualidade.

3.4 Parâmetros e Operadores Genéticos

Parâmetro	Valor
Tamanho da população	20 indivíduos
Taxa de cruzamento	80%
Taxa de mutação	3%
Elitismo	2 melhores indivíduos preservados
Máximo de gerações	1.000
Seleção de pais	Seleção por roleta (proporcional ao fitness)
Operador de cruzamento	Ponto de corte único
Operador de mutação	Bit flip aleatório

3.4.1 Seleção por Roleta (Roulette Wheel Selection)

Na seleção por roleta, cada indivíduo da população recebe uma fatia proporcional ao seu fitness na roda da sorte. A probabilidade de seleção do indivíduo i é dada por:

$$P(i) = \text{fitness}(i) / \sum_j \text{fitness}(j)$$

Um número aleatório uniforme em $[0, 1]$ é gerado e o indivíduo selecionado é aquele cuja fatia cumulativa contém esse valor. Indivíduos com fitness maior têm maior probabilidade de serem selecionados como pais, mas indivíduos com fitness menor ainda têm chance, mantendo a diversidade genética.

3.4.2 Cruzamento por Ponto de Corte (Single-Point Crossover) — Taxa: 80%

O cruzamento ocorre com probabilidade de 80% a cada par de pais selecionados. Um ponto de corte aleatório p ($1 \leq p \leq 23$) é escolhido no cromossomo de 24 bits. O primeiro filho herda os bits de posição 0 a $p-1$ do pai 1 e os bits de p a 23 do pai 2. O segundo filho recebe o

complemento. Se a probabilidade de cruzamento não for ativada, os filhos são cópias diretas dos pais.

3.4.3 Mutação por Bit Flip — Taxa: 3%

Após o cruzamento, cada bit do cromossomo do filho tem probabilidade de 3% de sofrer mutação. A mutação por bit flip simplesmente inverte o valor do bit: 0 torna-se 1 e 1 torna-se 0. Esta operação introduz variação genética e ajuda o algoritmo a escapar de ótimos locais, explorando regiões do espaço de busca não alcançáveis apenas pelo cruzamento.

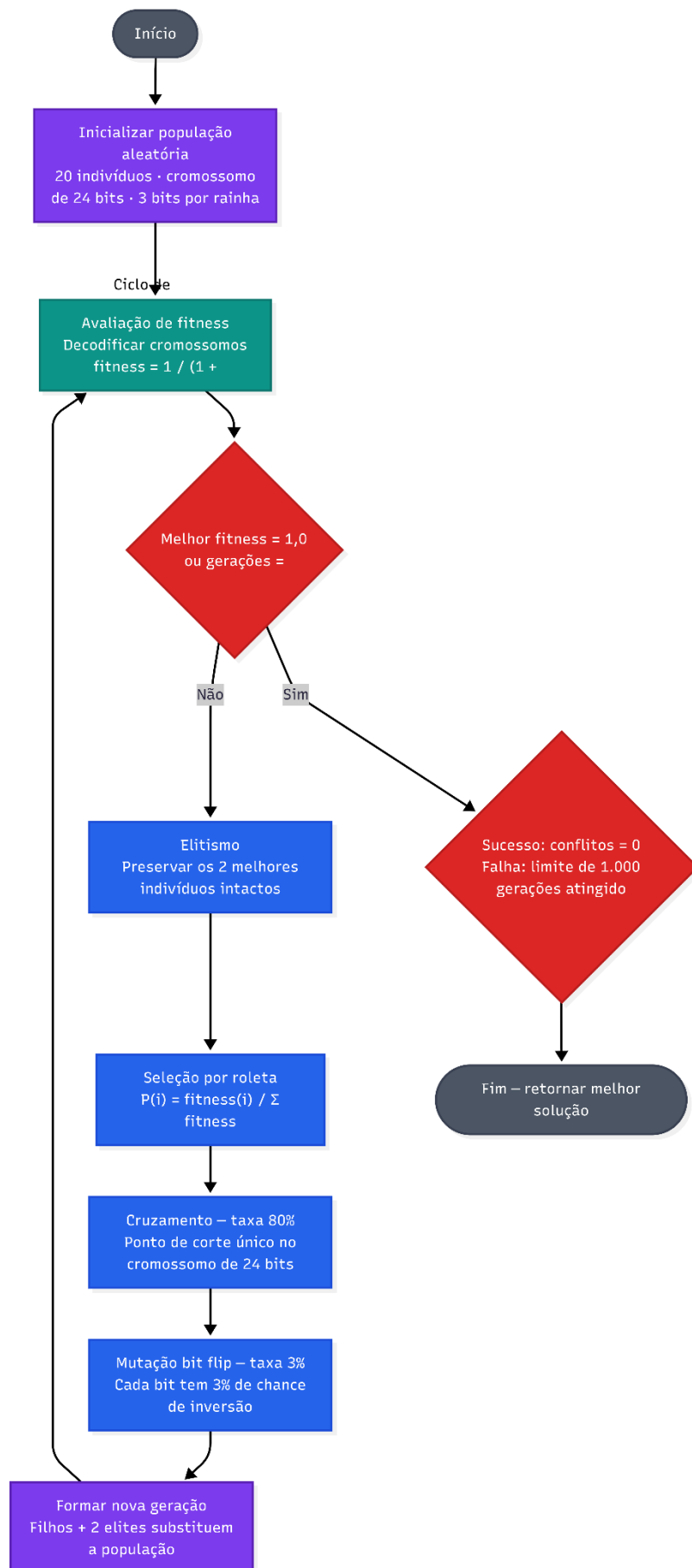
3.4.4 Elitismo — 2 melhores indivíduos

O elitismo preserva os 2 melhores indivíduos da geração atual na próxima geração, sem aplicação de cruzamento ou mutação. Isso garante que a melhor solução encontrada nunca seja perdida ao longo da evolução, o que melhora a monotonia do valor do melhor fitness ao longo das gerações.

3.5 Critérios de Parada

- Critério de ótimo: o algoritmo para antecipadamente quando um indivíduo com $\text{fitness} = 1,0$ ($C(Q) = 0$) é encontrado.
- Critério de recurso: o algoritmo para ao atingir 1.000 gerações, mesmo sem encontrar a solução ótima.

3.6 Fluxograma do Algoritmo Genético



3.7 Resultados Obtidos pelo Algoritmo Genético

O AG foi executado 50 vezes, obtendo os seguintes resultados:

- Taxa de sucesso: 76% — 38 das 50 execuções encontraram a solução ótima ($C(Q) = 0$); 12 execuções atingiram o limite de 1.000 gerações sem convergir.
- Tempo médio de execução: 217,32 ms (desvio padrão: 208,77 ms).
- Número médio de gerações até convergência: 397,22 (desvio padrão: 390,16).
- "Conflitos médios finais: 0,24 — média de $C(Q)$ sobre as 50 execuções, incluindo as 12 que não convergiram".
- As cinco melhores soluções distintas encontradas foram:

Rank	Vetor de posicionamento Q
1°	[6, 1, 5, 2, 0, 3, 7, 4]
2°	[2, 7, 3, 6, 0, 5, 1, 4]
3°	[1, 6, 2, 5, 7, 4, 0, 3]
4°	[4, 2, 0, 5, 7, 1, 3, 6]
5°	[5, 2, 4, 6, 0, 3, 1, 7]

Parte 4 — Automação Experimental com n8n

4.1 Visão Geral da Plataforma n8n

O n8n é uma plataforma de automação de fluxos de trabalho (workflows) de código aberto, baseada em uma interface visual de nós interconectados. Cada nó representa uma operação ou etapa de processamento, e os dados fluem entre eles de forma estruturada. A integração com o n8n foi essencial para garantir a reprodutibilidade dos experimentos, eliminar o hardcode de parâmetros, padronizar a coleta de métricas e automatizar o armazenamento dos resultados.

4.2 Estrutura do Workflow

O workflow desenvolvido no n8n é composto pelas seguintes etapas:

4.2.1 Nó de Configuração Global

O primeiro nó do workflow, denominado nó de configuração, define todas as variáveis globais de execução: N (número de rainhas), número de execuções (50), parâmetros do GRASP ($\alpha = 0,3$ e limite de iterações = 1.000) e parâmetros do AG (tamanho da população, taxas de cruzamento e mutação, elitismo e máximo de gerações). Essa abordagem elimina valores hardcoded nos algoritmos e centraliza a configuração, facilitando a modificação dos parâmetros para novos experimentos sem alteração do código dos algoritmos.

4.2.2 Nós de Execução Paralela

Dois nós de código independentes executam, em paralelo, as 50 repetições do GRASP e as 50 repetições do AG, respectivamente. Cada execução registra: o algoritmo utilizado, o tempo de execução em milissegundos, o número de iterações/gerações até a parada, o vetor de posicionamento da melhor solução encontrada, o valor final da função objetivo (número de conflitos) e um indicador booleano de sucesso (solução ótima encontrada ou não).

4.2.3 Consolidação dos Dados

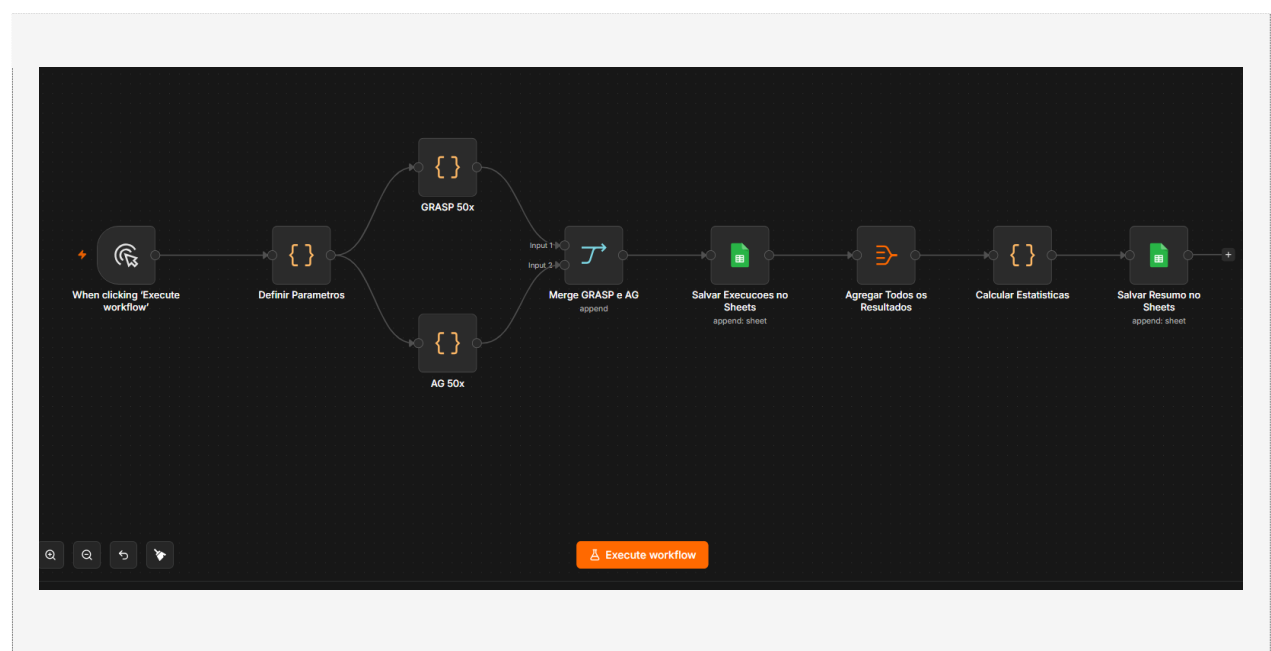
Os resultados das 100 execuções (50 por algoritmo) são consolidados por meio de nós de Merge e Aggregate. O nó de Merge combina os resultados dos dois algoritmos em um único fluxo de dados, enquanto o nó de Aggregate agrupa os resultados por algoritmo para facilitar o cálculo das estatísticas.

4.2.4 Cálculo de Estatísticas

Um nó de código dedicado calcula automaticamente todas as estatísticas matemáticas exigidas: média e desvio padrão do tempo de execução, média e desvio padrão do número de iterações/gerações, taxa de sucesso e fitness médio final para cada algoritmo.

4.2.5 Exportação para Google Sheets

Os dados brutos de cada execução e as estatísticas consolidadas são automaticamente exportados para planilhas do Google Sheets, que registram todos os campos exigidos pelo trabalho. A estrutura de armazenamento inclui uma aba de dados brutos (com todas as 100 execuções individuais) e uma aba de resumo estatístico, possibilitando análise futura e verificação da reprodutibilidade.



Parte 5 — Análise Experimental

5.1 Tabela Comparativa de Resultados

Métrica	GRASP	Algoritmo Genético
Tempo médio de execução	13,22 ms	217,32 ms
Desvio padrão do tempo	12,29 ms	208,77 ms
Média de iterações/gerações	6,9	397,22
Desvio padrão (iteraões)	6,43	390,16
Taxa de sucesso	100% (50/50)	76% (38/50)
Fitness médio final	0,00	0,24
Top 5 soluções distintas	[6,1,5,2,0,3,7,4] [3,1,6,2,5,7,4,0] [5,3,1,7,4,6,0,2] [2,5,7,0,4,6,1,3] [6,1,3,0,7,4,2,5]	[6,1,5,2,0,3,7,4] [2,7,3,6,0,5,1,4] [1,6,2,5,7,4,0,3] [4,2,0,5,7,1,3,6] [5,2,4,6,0,3,1,7]

5.2 Análise da Estabilidade dos Algoritmos

A estabilidade de um algoritmo pode ser avaliada pela relação entre a média e o desvio padrão das métricas observadas. O GRASP apresentou desvio padrão de tempo de 12,29 ms frente a uma média de 13,22 ms, resultando em um coeficiente de variação (CV) de aproximadamente 93%. Embora esse CV aparente alta variabilidade, é importante contextualizá-lo: todos os valores de tempo estão dentro de uma faixa estreita em termos absolutos (em torno de dezenas de milissegundos), e o algoritmo sempre convergiu para a solução ótima.

O AG, por sua vez, apresentou desvio padrão de 208,77 ms para uma média de 217,32 ms — também um CV elevado, porém com implicações mais sérias: as execuções malsucedidas, que atingiram o limite de 1.000 gerações sem convergir, contribuíram fortemente para esse desvio. As execuções bem-sucedidas do AG provavelmente convergem em um número de gerações consideravelmente menor, enquanto as fracassadas percorrem o caminho completo até o limite, gerando bimodalidade na distribuição dos tempos. Isso é evidenciado pelo desvio padrão de 390,16 gerações, próximo à própria média de 397,22 gerações.

Esse comportamento bimodal é característico de algoritmos populacionais com inicialização aleatória: quando a população inicial contém indivíduos de boa qualidade, a convergência ocorre rapidamente; quando a população inicial é desfavorável, o algoritmo pode ficar preso em regiões de baixa qualidade do espaço de busca por um longo período.

5.3 Velocidade de Convergência e Qualidade das Soluções

O domínio do GRASP sobre o AG é inequívoco nos dados experimentais obtidos. Em termos de velocidade de convergência, o GRASP requiriu em média apenas 6,9 iterações para encontrar a solução ótima, enquanto o AG necessitou de 397,22 gerações — uma razão de aproximadamente 1:58. Em termos de tempo de execução, a razão é ainda mais expressiva: 13,22 ms contra 217,32 ms, ou seja, o AG é aproximadamente 16,4 vezes mais lento que o GRASP.

Em relação à qualidade das soluções, ambos os algoritmos, quando convergem, encontram soluções igualmente ótimas ($C(Q) = 0$). No entanto, a taxa de sucesso de 76% do AG indica que em 24% das execuções o algoritmo falhou em encontrar a solução ótima dentro do limite de 1.000 gerações. O fitness médio final de 0,24 do AG (em contraste com 0,00 do GRASP) reflete essa ineficiência, pois as 12 execuções malsucedidas puxam a média para cima.

Vale destacar que a solução [6,1,5,2,0,3,7,4] foi encontrada por ambos os algoritmos como a melhor solução, indicando que se trata de uma das 92 soluções ótimas do Problema das 8 Rainhas e que ambos convergem para soluções verdadeiramente ótimas quando bem-sucedidos.

5.4 Influência da Aleatoriedade

A aleatoriedade desempenha papéis distintos nos dois algoritmos. No GRASP, ela é introduzida de forma controlada na fase de construção por meio do parâmetro α ($\alpha = 0,3$), que define o tamanho da RCL. Esse mecanismo garante que soluções iniciais diversificadas sejam construídas a cada iteração, evitando a estagnação em regiões específicas do espaço de busca, enquanto ainda garante qualidade mínima pela restrição da RCL. A busca local subsequente, por ser determinística, converte rapidamente essas soluções diversificadas em ótimos locais de boa qualidade.

No AG, a aleatoriedade está presente em múltiplos pontos: na inicialização da população, na seleção por roleta, nos pontos de cruzamento e nos bits sujeitos à mutação. Essa aleatoriedade multi-nível é ao mesmo tempo um ponto forte (permite exploração ampla do espaço de busca) e um ponto fraco (pode resultar em destruição de soluções boas durante o cruzamento ou convergência prematura para ótimos locais). Para o Problema das 8 Rainhas,

com $N=8$ e espaço de busca relativamente pequeno, a alta frequência de operações aleatórias parece ser mais prejudicial do que benéfica.

5.5 Custo Computacional e Análise das Falhas do AG

O custo computacional observado favorece amplamente o GRASP: tempo total para 50 execuções de aproximadamente 661 ms versus 10.866 ms para o AG. As 12 falhas do AG (execuções que atingiram o limite de 1.000 gerações) podem ser atribuídas a três fenômenos bem conhecidos na literatura de AGs:

- **Convergência prematura:** a população perde diversidade genética rapidamente, com todos os indivíduos convergindo para o mesmo ótimo local subótimo, tornando o cruzamento ineficaz.
- **Epistasia:** interações complexas entre os bits do cromossomo podem dificultar a melhora incremental por cruzamento de ponto de corte.
- **Deceptividade:** o fitness pode aumentar em direções que se afastam da solução ótima global, induzindo a busca para regiões subótimas.

Parte 6 — Discussão e Comparação

6.1 Diferenças Estruturais entre GRASP e Algoritmo Genético

As diferenças de desempenho observadas têm raízes profundas nas escolhas estruturais de cada algoritmo. O GRASP é uma metaheurística baseada em trajetória única (single-solution): cada iteração parte de uma solução recém-construída e a melhora de forma determinística. Essa abordagem é altamente eficiente para problemas onde a busca local é eficaz, pois concentra todo o esforço computacional em uma única solução de cada vez.

O AG, em contraste, é uma metaheurística baseada em população: mantém 20 soluções simultâneas e aplica operadores de recombinação para explorar o espaço de busca. Essa abordagem é teoricamente mais robusta para espaços de busca multimodais complexos, mas introduz overhead computacional significativo na manutenção e evolução da população. Para problemas simples como $N=8$, esse overhead não é compensado por ganhos de qualidade.

Essa diferença estrutural é fundamental: o GRASP com busca local eficaz é um intensificador poderoso, enquanto o AG é primordialmente um explorador diversificado. Para o Problema das 8 Rainhas com $N=8$, a intensificação supera a exploração em eficácia, explicando a supremacia do GRASP.

6.2 Limitações de Cada Abordagem

Limitações do GRASP para este problema:

- A ausência de memória entre iterações implica que o GRASP pode redescobrir soluções já encontradas, sem aprender com o histórico de busca.
- Para instâncias maiores ($N = 50, 100$), a eficiência da busca local pode degradar substancialmente, pois o espaço de vizinhança cresce quadraticamente.
- A escolha do parâmetro α não foi otimizada de forma sistemática, podendo haver valores mais eficientes para instâncias maiores.

Limitações do AG para este problema:

- A codificação binária com 3 bits por rainha é inadequada para o Problema das 8 Rainhas: valores de 0 a 7 são representados, mas o cruzamento e a mutação operando no nível dos bits não respeitam a semântica das posições das rainhas, gerando soluções inválidas frequentes que prejudicam a evolução.
- A taxa de mutação de 3% pode ser insuficiente para manter a diversidade genética com populações de apenas 20 indivíduos, levando à convergência prematura.

- A seleção por roleta pode sofrer de pressão seletiva excessiva quando um indivíduo possui fitness muito superior aos demais, monopolizando a reprodução.

6.3 Comparação Final

Com base nos resultados experimentais obtidos, o GRASP demonstrou ser nitidamente superior ao Algoritmo Genético para o Problema das 8 Rainhas com $N=8$ em todos os critérios avaliados: taxa de sucesso (100% vs. 76%), tempo médio de execução (13,22 ms vs. 217,32 ms), número de iterações/gerações até convergência (6,9 vs. 397,22) e estabilidade dos resultados. Esses resultados são coerentes com a literatura especializada, que aponta que metaheurísticas baseadas em trajetória com busca local eficaz frequentemente superam algoritmos evolutivos em problemas de baixa dimensionalidade.

É importante ressaltar que esses resultados não devem ser generalizados como indicativos de superioridade intrínseca do GRASP sobre AGs. Para problemas de alta dimensionalidade, multiobjetivo ou com espaços de busca altamente multimodais, os AGs tendem a demonstrar vantagens significativas devido à sua capacidade de exploração em paralelo.

6.4 Sugestões de Melhorias Futuras

Para o Algoritmo Genético:

- Representação permutacional: substituir a codificação binária por representação direta de permutações (codificação inteira), eliminando soluções inválidas e tornando os operadores de cruzamento (como OX — Order Crossover) semanticamente coerentes com o problema.
- Adaptação dinâmica da taxa de mutação: aumentar a taxa de mutação progressivamente quando a diversidade da população cai abaixo de um limiar, prevenindo a convergência prematura.
- Mutação por inversão de subsegmento: em vez de bit flip aleatório, aplicar inversão de subsegmentos do vetor de posições, que preserva mais informação estrutural.
- Tuning fino de hiperparâmetros: realizar busca sistemática (grid search ou busca bayesiana) pelos valores ótimos de taxa de cruzamento, taxa de mutação e tamanho de população.
- Aumento do tamanho da população: uma população maior (p. ex., 50 ou 100 indivíduos) aumentaria a diversidade genética e reduziria a probabilidade de convergência prematura.

Para o GRASP:

- Memória adaptativa: incorporar estruturas de memória semelhantes à Busca Tabu para evitar revisitar regiões já exploradas.
- Análise da sensibilidade ao alpha: realizar experimentos variando alpha sistematicamente (0,1; 0,2; ...; 0,9) para identificar o valor ótimo para diferentes tamanhos de N.
- Busca local mais elaborada: substituir a busca local por trocas de pares por operadores mais sofisticados, como a busca em vizinhança variável (VNS), potencialmente aumentando a qualidade para instâncias maiores.

6.5 Conclusão

Este trabalho demonstrou de forma empírica e analítica as diferenças de desempenho entre o GRASP e o Algoritmo Genético no contexto do Problema das 8 Rainhas. A automação experimental com o n8n garantiu reprodutibilidade e rigor metodológico na coleta de dados. Os resultados evidenciam que a escolha da metaheurística adequada depende fundamentalmente das características do problema: dimensionalidade, estrutura do espaço de busca, disponibilidade de operadores de melhora local eficientes e restrições de tempo computacional. Para problemas de menor escala com estrutura favorável à busca local, o GRASP se mostrou uma escolha superior. Para problemas de maior complexidade e dimensionalidade, espera-se que o AG, com as melhorias sugeridas, apresente vantagens significativas.